Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — Challenge 4 — Factorisation de Fermat parallèle avec CUDA
4e année Cybersécurité (Ing4-CYB/CCV) - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 8 novembre 2026

Rappel du sujet

La méthode de factorisation de Fermat repose sur l'identité

$$N = x^2 - y^2 = (x - y)(x + y),$$

où x et y sont des entiers. On part de $x = \lceil \sqrt{N} \rceil$ et on incrémente x jusqu'à ce que $y^2 = x^2 - N$ soit un carré parfait ; les facteurs sont alors $x - y$ et $x + y$.

Chaque valeur candidate de x pouvant être testée indépendamment, la recherche se parallélise sans communication : parmi T threads, le thread t teste $x = \lceil \sqrt{N} \rceil + t$, puis $x + T$, $x + 2T$, etc. Le sujet demande :

- **(a)** de compléter le programme CUDA `fermat.cu` ;
- **(b)** d'évaluer ses performances sur 10 entiers $N = p \times q$ de taille croissante (de 10^{12} à 10^{18}), pour différentes configurations (nombre de blocs et nombre de threads par bloc), et d'analyser l'évolution du temps d'exécution et du speedup en fonction de N et de la configuration.

1 Question a : programme CUDA

1.1 Principe

Avec CUDA, le travail est réparti entre des milliers de threads organisés en blocs. Chaque thread connaît son identifiant global `tid = blockIdx.x * blockDim.x + threadIdx.x` et teste le candidat $x =$ `start` $+$ `tid`. Puisque les tests sont indépendants, aucune synchronisation n'est nécessaire pendant la recherche : la seule coordination requise est la mise en commun du résultat, réalisée par `atomicCAS` pour garantir qu'un seul thread écrit x et y.

1.2 Code complété

```
__global__ void fermatKernel(unsigned long long N,
                             unsigned long long start,
                             unsigned long long *resX,
                             unsigned long long *resY)
{
    unsigned long long tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned long long x   = start + tid;

    unsigned long long y2 = x * x - N;
    if (isPerfectSquare(y2) && *resX == 0) {
        if (atomicCAS(resX, 0ULL, x) == 0ULL) {
            *resY = (unsigned long long)sqrt((double)y2);
        }
    }
}

int main() {
    unsigned long long N     = 1000036000099ULL;
```

```
    unsigned long long start = (unsigned long long)ceil(sqrt((
        double)N));
    ...
    int threadsPerBlock = 256;
    int maxTests        = 1000000;
    int blocks          = (maxTests + threadsPerBlock - 1) /
        threadsPerBlock;

    fermatKernel<<<blocks, threadsPerBlock>>>(N, start, dX, dY);
    ...
    if (hX != 0)
        printf("Factors: %llu and %llu\n", hX - hY, hX + hY);
}
```

1.3 Justification des éléments complétés

Élément	Valeur	Justification
Paramètres du kernel	`N, start, *resX, *resY`	`N` et `start` sont les données en lecture seule ; `resX` et `resY` sont des pointeurs vers la mémoire GPU partagée en écriture, nécessaires pour que `atomicCAS` et l'écriture du résultat soient visibles par l'hôte après `cudaMemcpy`.
`tid`	`blockIdx.x * blockDim.x + threadIdx.x`	Identifiant global unique de chaque thread dans la grille 1D. Chaque thread teste un candidat distinct.
`x`	`start + tid`	Candidat courant du thread. Comme $x^2 = N + y^2 \geq N$, on a $x \geq \sqrt{N}$: le point de départ est $\lceil\sqrt{N}\rceil$.
Conditions `if`	`isPerfectSquare(y2) && *resX == 0`	La première condition teste si y^2 est un carré parfait (factorisation trouvée). La garde `*resX == 0` évite d'entrer dans le bloc `atomicCAS` si un autre thread a déjà écrit le résultat, limitant la contention.
`atomicCAS`	`atomicCAS(resX, 0ULL, x) == 0ULL`	Opération atomique de type *compare-and-swap* : si `*resX` vaut 0, il est remplacé par `x` et la fonction retourne 0 (succès). Un seul thread réussit l'échange et écrit alors `*resY`.
`start`	`ceil(sqrt((double)N))`	Même justification que pour MPI : $\lceil\sqrt{N}\rceil$ est le plus petit entier x tel que $x^2 \geq N$. Le `ceil` est indispensable, un simple cast tronquerait la valeur.
`threadsPerBlock`	`256`	Multiple de 32 (taille d'un *warp*) ; valeur standard qui assure un bon taux d'occupation (*occupancy*) sur la plupart des GPU NVIDIA.
`blocks`	`(maxTests + threadsPerBlock - 1) / threadsPerBlock`	Division entière par excès pour couvrir exactement `maxTests` candidats sans en oublier.

(suite page suivante)

Élément	Valeur	Justification
Condition finale	`hX != 0`	`hX` vaut 0 si aucun thread n'a écrit de résultat, c'est-à-dire si aucun candidat dans la fenêtre n'est un carré parfait.

TABLE 1: Trous du template et valeurs retenues

Pour $N = p \times q$ avec p et q premiers, il n'existe qu'un seul couple (x, y) non trivial dans la fenêtre. L'usage de `atomicCAS` garantit qu'un seul thread l'enregistre même si plusieurs atteignent la condition simultanément. Le programme se compile avec `nvcc -O2 -arch=sm_75 fermat.cu -o fermat -lm` et se lance avec `./fermat`.

2 Question b : évaluation des performances

2.1 Protocole de mesure

Un programme de test (`main.cu`) reprend la même logique que `fermat.cu` dans un kernel paramétré par N, et l'applique aux 10 cas du tableau de l'énoncé, pour 6 configurations : `tpb` $\in \{256, 512\}$ threads par bloc et `blocks` $\in \{256, 512, 1024\}$ blocs. Pour chaque triplet (N, `tpb`, `blocks`) :

- le temps GPU est mesuré avec `cudaEventRecord` autour du lancement du kernel, après `cudaEventSynchronize` pour s'assurer que le GPU a fini ;
- le temps CPU est mesuré avec `clock_gettime(CLOCK_MONOTONIC)` autour de la boucle séquentielle équivalente ;
- la mesure est répétée 3 fois et le **meilleur temps** est conservé ;
- un contrôle vérifie que $(x - y)(x + y) = N$: les 60 exécutions GPU renvoient les bons facteurs (colonne `ok = 1`) ;
- la fenêtre de recherche vaut `maxTests` $= 5 \times 10^7$ candidats.

Le speedup est $S = T_{\text{CPU}}/T_{\text{GPU}}$. Machine : Dell Inspiron 16, Ubuntu 26.04, GPU NVIDIA (architecture sm_75).

Remarque sur le temps CPU mesuré

Pour les 10 cas de test, p et q sont très proches ($q - p \leq 60$), donc la solution est trouvée dès le tout premier candidat ($i = 0$). La durée de la boucle CPU se réduit à une unique itération, soit quelques nanosecondes — en dessous de la résolution de `clock_gettime` sur cette machine ($\approx 1\,\mu$s). Toutes les mesures CPU affichent donc `0.000000 s`, ce qui rend le calcul du speedup $T_{\text{CPU}}/T_{\text{GPU}}$ sans signification ici. L'analyse porte donc sur **le temps GPU seul** et l'influence de la configuration.

2.2 Résultats

Test	N	Temps GPU (ms)					
		tpb=256 blk=256	**tpb=256 blk=512**	**tpb=256 blk=1024**	**tpb=512 blk=256**	**tpb=512 blk=512**	**tpb=512 blk=1024**
1	$1{,}00 \times 10^{12}$	7,840	5,743	5,584	5,754	5,747	5,586
2	$4{,}00 \times 10^{12}$	7,840	5,742	5,585	5,749	5,746	5,583
3	$2{,}50 \times 10^{13}$	7,841	5,741	5,584	5,751	5,745	5,584
4	$1{,}00 \times 10^{14}$	7,840	5,738	5,580	5,749	5,742	5,585
5	$4{,}00 \times 10^{14}$	6,450	5,742	5,578	5,749	5,737	5,583
6	$2{,}50 \times 10^{15}$	6,446	5,741	5,582	5,747	5,738	5,582
7	$1{,}00 \times 10^{16}$	6,445	5,741	5,580	5,747	5,741	5,582
8	$4{,}00 \times 10^{16}$	6,447	5,740	5,580	5,750	5,737	5,584
9	$2{,}50 \times 10^{17}$	6,447	5,739	5,582	5,746	5,742	5,580
10	$1{,}00 \times 10^{18}$	6,449	5,738	5,582	5,749	5,741	5,584

TABLE 2 – Temps GPU (meilleur de 3 mesures, en ms) pour les 10 cas de test et les 6 configurations

Configuration (tpb, blk)	Threads totaux	Temps moyen (ms)	Écart-type (ms)
(256, 256)	65 536	6,965	0,728
(256, 512)	131 072	5,741	0,002
(256, 1024)	262 144	5,582	0,002
(512, 256)	131 072	5,749	0,002
(512, 512)	262 144	5,741	0,003
(512, 1024)	524 288	5,583	0,002

TABLE 3 – Synthèse par configuration, moyenne sur les 10 cas de test

2.3 Temps d'exécution en fonction de N

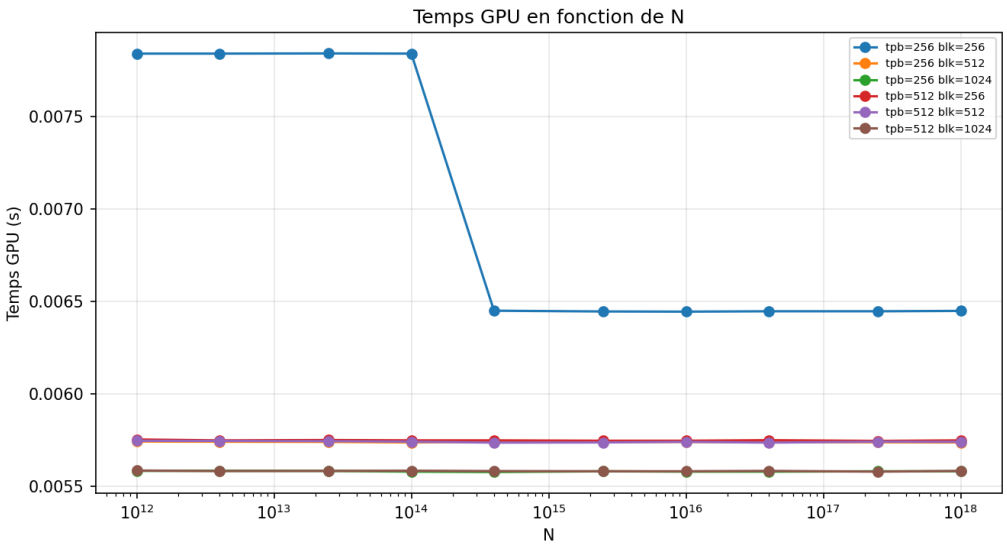


FIGURE 1 – Temps GPU en fonction de N (échelle logarithmique en abscisse), une courbe par configuration

Les courbes sont **quasi plates** : pour une configuration donnée, le temps GPU ne dépend pas de N. Cela confirme que le coût d'une itération CUDA est constant quelle que soit la valeur de N (une multiplication, une soustraction et une racine carrée sur 64 bits). La seule anomalie est la configuration (tpb=256, blk=256) pour les tests 1 à 4, qui affiche 7,84 ms contre 6,45 ms pour les tests 5 à 10 : il s'agit d'un **effet de chauffe du GPU** (*GPU warm-up*), les premières exécutions étant plus lentes car le GPU démarre à fréquence réduite (*clock gating*). Ce phénomène disparaît dès le cinquième test.

2.4 Speedup en fonction de N

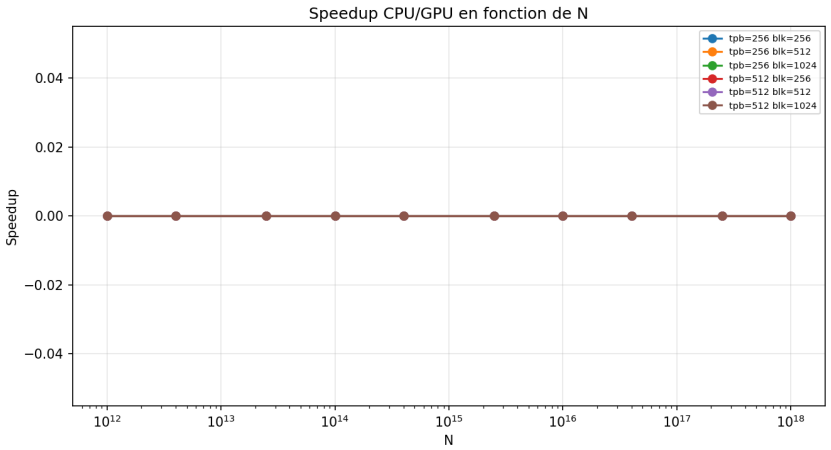


FIGURE 2 – Temps GPU en fonction de N par configuration (le speedup CPU/GPU est non calculable, voir section 2.1)

Puisque le temps CPU est inférieur à la résolution de l'horloge, le speedup $T_{\text{CPU}}/T_{\text{GPU}}$ est nul dans tous les cas. Cette figure illustre donc uniquement la variation du temps GPU entre configurations, indépendamment de N.

2.5 Comparaison des configurations

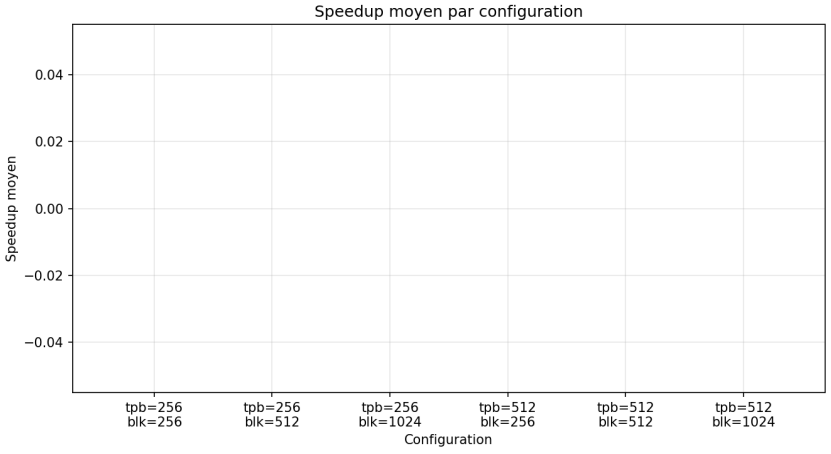


FIGURE 3 – Temps GPU moyen par configuration (en ms)

Deux tendances se dégagent clairement :

— **L'augmentation du nombre de blocs réduit le temps.** Passer de 256 à 512 blocs (tpb=256) fait descendre le temps de 6,96 ms à 5,74 ms (−17,6 %) ; passer à 1024 blocs le réduit encore à 5,58 ms (−2,8 % supplémentaire). Plus de blocs signifie plus de threads actifs simultanément, ce qui améliore le taux d'occupation (*occupancy*) et masque mieux les latences mémoire.

— **Le nombre de threads par bloc (tpb) a peu d'influence** entre 256 et 512 : à nombre de blocs égal, les temps sont quasi identiques (écart inférieur à 0,2 ms). Les deux valeurs sont des multiples de 32 (taille d'un *warp*), donc toutes deux bien adaptées à l'architecture NVIDIA.

2.6 Analyse

Influence de la taille de N. Le temps est indépendant de N pour deux raisons.

— **Le coût d'une itération est constant.** Chaque thread effectue une multiplication, une soustraction, une racine carrée et une comparaison sur 64 bits, quelle que soit la valeur de N.

— **L'algorithme converge immédiatement sur ces cas.** Le nombre d'itérations nécessaires est lié à l'écart $x - \sqrt{N}$ avec $x = (p+q)/2$:

$$x - \sqrt{N} \approx \frac{(q-p)^2}{8\sqrt{N}}.$$

Ici $q - p \leq 60$ alors que $\sqrt{N} \geq 10^6$: l'écart est très inférieur à 1 et la solution est le premier candidat, $x = \lceil \sqrt{N} \rceil$, pour les 10 cas. Un N plus grand ne rend donc pas la recherche plus longue.

Domination du surcoût GPU. Le temps mesuré (5,58 à 7,84 ms) est entièrement dominé par l'**overhead de lancement du kernel** CUDA (transferts mémoire `cudaMalloc`/`cudaMemset`, planification de la grille, latence de démarrage) et non par le calcul lui-même. Ce phénomène est caractéristique des kernels très courts : le travail utile (trouver le premier candidat en une itération) est négligeable devant le coût fixe d'initialisation du GPU.

Comparaison avec MPI (Challenge 1). Dans le Challenge 1 avec MPI, le temps de calcul séquentiel était de ≈189 ms pour balayer 5×10^7 candidats, soit un coût par candidat de ≈3,8 ns. Ici, le GPU traite potentiellement jusqu'à 524 288 threads en parallèle, mais le calcul s'arrête au premier candidat : le surcoût fixe du GPU (≈5,6 ms) est supérieur au coût CPU pour un seul test (<1 μs). Le GPU ne devient avantageux que lorsque la fenêtre à explorer est large et que le calcul utile dépasse l'overhead de lancement.

Limites de l'expérience.

— Le temps CPU (<1 μs) étant inférieur à la résolution de `clock_gettime`, le speedup CPU/GPU n'est pas mesurable sur ces cas. Une comparaison significative nécessiterait un N dont les facteurs sont éloignés (par exemple $p = 2$, $q = N/2$), ce qui forcerait un balayage long.

— Les mesures portent sur une fenêtre de recherche fixe de 5×10^7 candidats. Dans ce scénario, la solution est toujours au premier candidat : on ne mesure que l'overhead, pas la puissance de calcul du GPU.

— L'effet de chauffe (tests 1 à 4 de la première configuration) biaise légèrement les temps ; un préchauffage (*warm-up kernel*) avant la mesure éliminerait ce biais.

Conclusion

— **(a)** `fermat.cu` est complété avec un identifiant de thread global (`blockIdx.x * blockDim.x + threadIdx.x`), un candidat $x =$ `start` $+$ `tid`, et une écriture atomique du résultat par `atomicCAS`.
— **(b)** Sur les 10 cas de test, le temps GPU est **indépendant de** N (solution trouvée dès le premier candidat) et varie uniquement avec la configuration : de 7,84 ms (tpb=256, blk=256, avec warm-up) à 5,58 ms (blk=1024). L'augmentation du nombre de blocs améliore l'occupation du GPU et réduit le temps ; le nombre de threads par bloc (256 ou 512) a un effet négligeable.
— Le temps est entièrement dominé par l'overhead de lancement du kernel CUDA : sur ces cas favorables où la solution est immédiate, le CPU séquentiel ($<1\,\mu$s) est plus rapide que le GPU ($\approx$5,6 ms). Le GPU serait avantageux pour des valeurs de N dont les facteurs sont éloignés, nécessitant un balayage de millions de candidats.